The solution is included in `CH03/03 - Exercise.ipynb`, which is explained as follows.

1. We first import the necessary libraries: PyTorch for array/tensor manipulation, GPyTorch for GP modeling, and Matplotlib for visualization.

```
import torch
import gpytorch
import matplotlib.pyplot as plt
```

   We then implement the objective function using the given formula.

```
def f(x):
    return (
        torch.sin(5 * x[..., 0] / 2 - 2.5) * torch.cos(2.5 - 5 * x[..., 1])
        + (5 * x[..., 1] / 2 + 0.5) ** 2 / 10
    ) / 5 + 0.2
```

2. To visualize the function, we need to create a mesh grid the domain. We store this grid in `xs`.

```
lb = 0
ub = 2
xs = torch.linspace(lb, ub, 101) ❶
x1, x2 = torch.meshgrid(xs, xs)
xs = torch.vstack((x1.flatten(), x2.flatten())).transpose(-1, -2) ❷
```

   ❶ One-dimensional grid.

   ❷ Two-dimensional grid.

   We then can obtain the function values over this grid by passing `xs` to `f()`. The results are stored in `ys`.

```
ys = f(xs)
```

   We use `plt.imshow()` to visualize `ys` as a heat map.

```
plt.imshow(ys.reshape(101, 101).T, origin="lower", extent=[lb, ub, lb, ub])
```

   This will produce Figure A.2.

3. To randomly sample 100 points within the domain, we use `torch.rand()` to sample from the unit square, and then multiply the result by 2 to scale it to our domain.

```
torch.manual_seed(0)
train_x = torch.rand(size=(100, 2)) * 2
```

   The function values of these points can be obtained by calling `f(train_x)`.

```
train_y = f(train_x)
```

4. We implement our GP model as specified.

```
class GPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ConstantMean()
        self.covar_module = gpytorch.kernels.ScaleKernel(
            gpytorch.kernels.MaternKernel(
                nu=2.5,
                ard_num_dims=None ❶
            )
        )

    def forward(self, x):
        mean_x = self.mean_module(x)
        covar_x = self.covar_module(x)
        return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)
```

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>